



ToonyVoices

Table of Contents

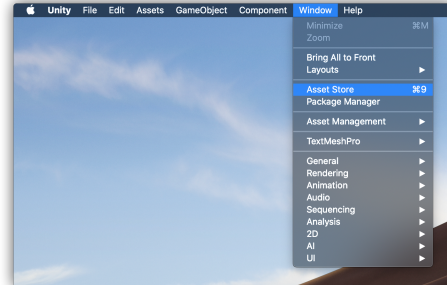
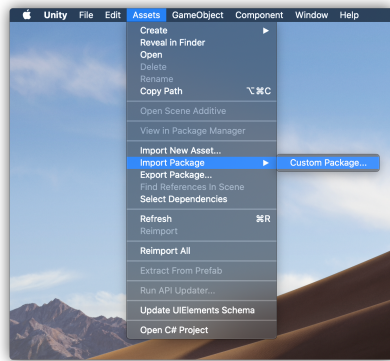
- Introduction
 - Setup and Walk-through
 - Usage
 - Special Notes
 - Credits
-

Introduction

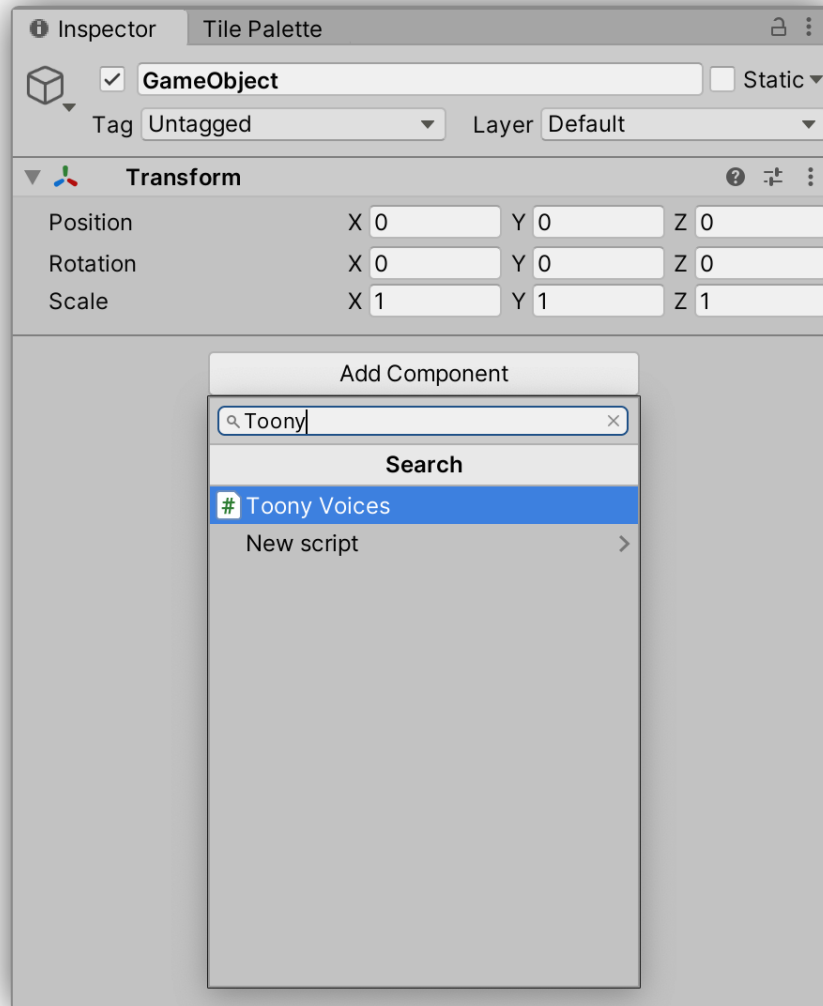
ToonyVoices is a game audio asset, providing dynamically created cartoon voice sounds similar to the Animal Crossing series of games, with only a single line of code!

Setup and Walk-through

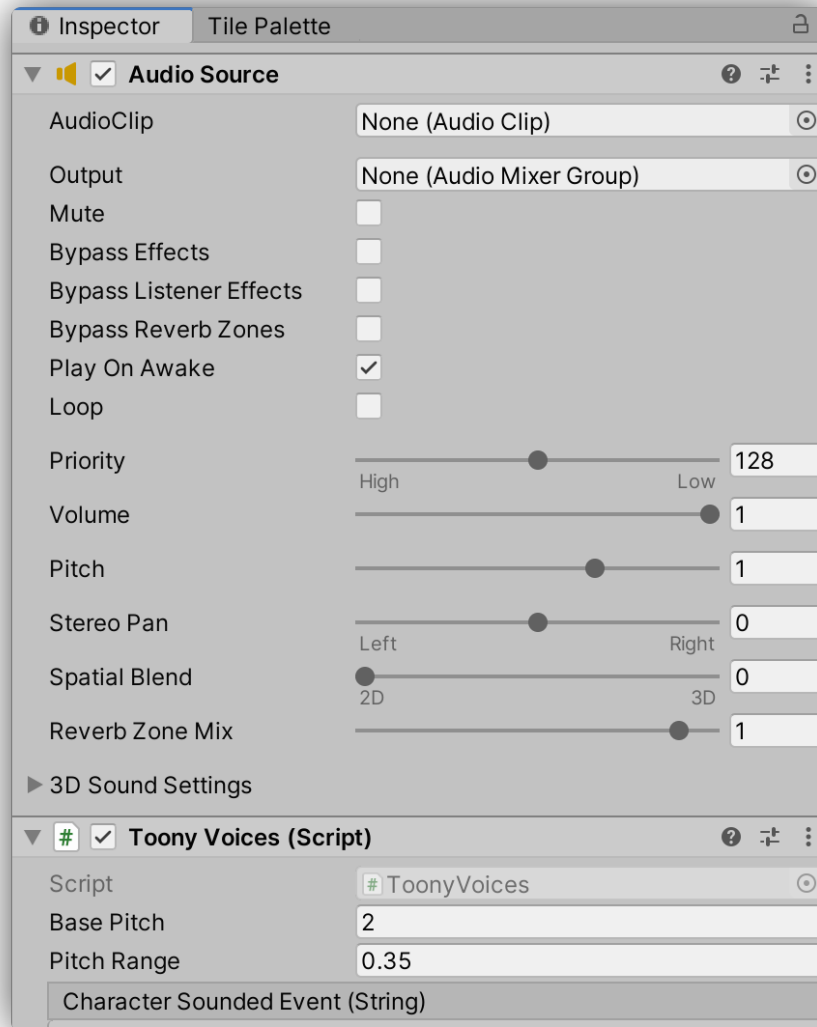
- Import the *ToonyVoices.unitypackage* into your project. This will either be done with an individually downloaded file, or through the Unity Asset Store from the editor.



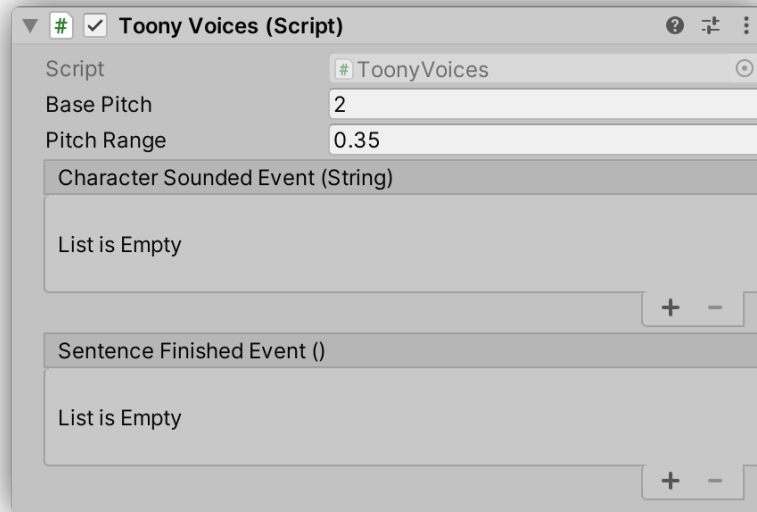
- Select the object from the hierarchy that you want to be able to speak.
- From the inspector, scroll down and click the *Add Component* button, and attach the **ToonyVoices** component.



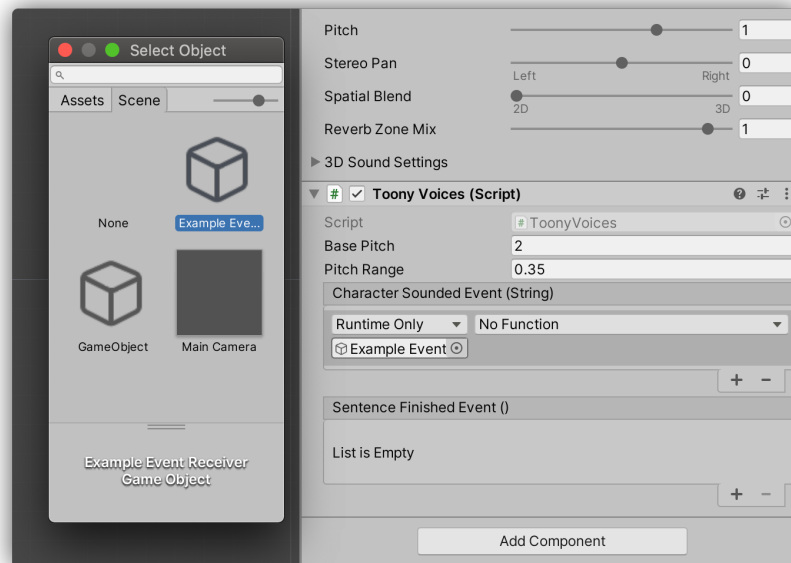
- You will now see that your object has not only the **ToonyVoices** component attached, but also an **AudioSource**. This is required for the **ToonyVoices** component to work, so we removed an extra step for you and made it attach automatically!



- Looking over the **ToonyVoices** component in the inspector, you will see two fields that accept a `float` value. One for the base pitch, and the other for the pitch variation $\pm$ between each character spoken. There are default values in place for each of these properties, which sound pretty good by default, but they are there to be tweaked and played with! Just do not forget, if the values are changed while in the editors *Play Mode*, they will not be saved when exiting *Play Mode*.

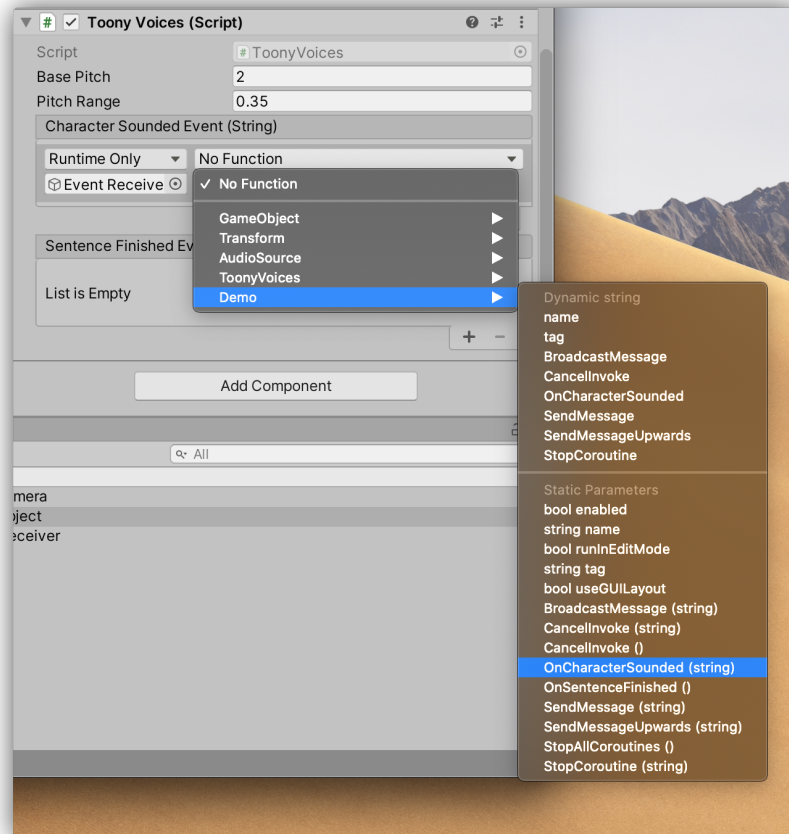


- Below those two fields for changing the pitch values, you can see two boxes for *Character Sounded Event (String)* and *Sentence Finished Event*. Both of these fields will be wanting a *GameObject* attached to it from the hierarchy, and that can be done by either dragging and dropping the object, or clicking on the button that looks like a circle with a dot in it; this pops up a list of objects for you to select from.



- When filling out the details for your event listener, the *Character*

Sounded Event (String) field is expecting to have a method that accepts a **string** type so it will know what the last sounded character was. In the provided demo, we provided the method `OnCharacterSounded(string)`, simply to keep things simple for the demo. With the second field, *Sentence Finished Event*, your method does not need to accept any fields. As with our last example, we named ours `OnSentenceFinished`.



Usage

Basic speech invocation

```
// On the attached GameObject
GetComponent<ToonyVoices>().Speak("Hello, world!");
```

We were not kidding when saying it only took a single line of code to have your characters speaking! Now of course, in all practical senses you would want to

create a reference to the `ToonyVoices` object for future use, instead of having to fetch the component each time. But even that is simple!

```
public class ExampleClass : MonoBehaviour
{
    [...]
    ToonyVoices voices;

    public void Start()
    {
        [...]
        // Get the ToonyVoices component
        voices = GetComponent<ToonyVoices>();

        // Time to say something
        SayHello();
    }

    public void SayHello()
    {
        // What will we be saying
        voices.Speak("Hello, world!");
    }
}
```

Now do not worry that the code snippet has grown so large, most of this will already be in place within your script! Truth be told, we have only added two additional lines of code, and improved performance by not having to call `GetComponent<ToonyVoices>()` every time we need to use it.

Now let us say we want to have an entity respond to our first bit of speech though a bit more excited, as they have just been greeted! The problem is, we have already dialed in the perfect pitch for their character, and it would be nice not to have to mess with that. Well, there is an overloaded method just for that!

```
voices.Speak("Well hello to you too!", 3f);
```

This method simply accepts a pitch `float` value to be used for the current sentence, and afterward, it reverts to the base pitch set within the Inspector. Nothing too terrible to fret about.

You can also simulate a whisper, by expanding on that overloaded method we just used. It only takes the addition of another float, and we can control the volume of the sentence as well. Combine that with a bit lower pitch, and we have ourselves a whisper! Shhh!

```
voices.Speak("Quiet voices please.", 2f, 0.6f);
```

Handling speech events

That is it for the speaking portion of the **ToonyVoices** class, but **ToonyVoices** also has another powerful feature baked into it, the power of Unity Events! The events were touched on in the Setup and Walk-through section, and configuring them within the Inspector, but let's take a look at how to use the events entirely from code.

Let's take a look at the example class from before, and just build on it a little bit. Firstly, it's time to outline what we will be doing.

To start let's add a method that will simply log a message to the console after a sentence has finished being spoken, but to do that we need to first use the **AddListener** method of the **ToonyVoices.SentenceFinished** property and give it a method to call whenever the event is fired. It would also be good to clean up after ourselves when we are finished, and thankfully we can use the **RemoveListener** method when we are done; the code could look something like this.

```
public class ExampleClass : MonoBehaviour
{
    [...]
    ToonyVoices voices;

    public void Start()
    {
        [...]
        // Get the ToonyVoices component
        voices = GetComponent<ToonyVoices>();

        // Add the event listener
        voices.SentenceFinished.AddListener(OnSentenceFinished);

        // Time to say something
        SayHello();
    }

    public void OnDestroy()
    {
        // Cleanup our event listener
        voices.SentenceFinished.RemoveListener(OnSentenceFinished);
    }

    [...]

    public void OnSentenceFinished()
    {
```

```

        // Write a little message to the console
        Debug.Log("The SentenceFinished event has fired!");
    }
}

```

Truly, we had to add very little to bring events into our class. If we break it down, we only had to add three lines of code outside of our method definitions, not bad! Taking a bit deeper look into the changes, we created an `OnSentenceFinished` method that will be called every time our `voices.Speak` call wraps up, it does not do much at this point but your imagination is the limit! We also registered our method with `AddListener` and `RemoveListener` in the `Start` and `OnDestroy` methods respectively, without doing this our `SentenceFinished` event would not know what to call.

This just leaves us with one more event we need to cover, and that is the `CharacterSounded` event. As you may have guessed it gets called every time a character, in most instances a letter, is spoken; the characters include numbers, symbols, and punctuation too!

Once again we are going to build upon the previous code sample, much in the same way by adding a new method and registering it with the `CharacterSounded` event by calling `AddListener` and `RemoveListener`. The only real difference is the method we will be creating needs to accept a `string` value as a parameter, this way we can know what character has just sounded. So let's dive in!

```

public class ExampleClass : MonoBehaviour
{
    [...]
    ToonyVoices voices;

    public void Start()
    {
        [...]
        // Get the ToonyVoices component
        voices = GetComponent<ToonyVoices>();

        // Add the event listeners
        voices.SentenceFinished.AddListener(OnSentenceFinished);
        voices.CharacterSounded.AddListener(OnCharacterSounded);

        // Time to say something
        SayHello();
    }

    public void OnDestroy()
    {
        // Cleanup our event listeners
    }
}

```

```

        voices.SentenceFinished.RemoveListener(OnSentenceFinished);
        voices.CharacterSounded.RemoveListener(OnCharacterSounded);
    }

    [...]

    public void OnSentenceFinished()
    {
        // Write a little message to the console
        Debug.Log("The SentenceFinished event has fired!");
    }

    public void OnCharacterSounded(string character)
    {
        // Write the character that was sounded to the console
        Debug.Log("The character that sounded was: " + character);
    }
}

```

We created our `OnCharacterSounded` method right under the method from the last section, and the same with calling `AddListener` and `RemoveListener` on the `CharacterSounded` event. Now whenever that event fires, you will have access to the character, as a string, that was last spoken within your method. We have not done anything fancy with that character in this example, but you could take that and have it added to a speech bubble as they are spoken, or anything else that your mind can dream up!

Special Notes

Compound sounds

When receiving handling `CharacterSounded` events, it may be important to note that *sh* and *th* are given their own sounds; as such you will receive both characters in an event call when they are found next in the sentence.

Script Execution Order

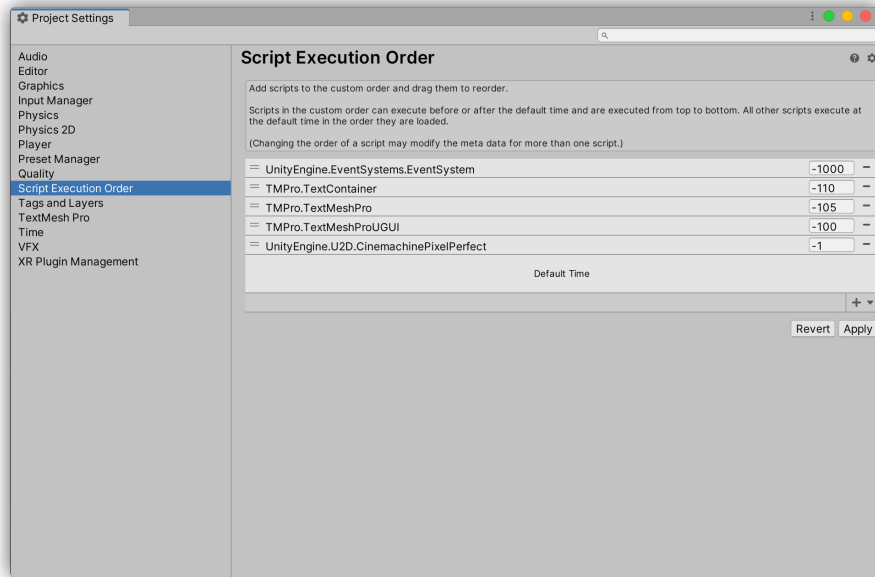
There is a chance that you may experience an `IndexOutOfRangeException` being thrown when you try to run your project after just having setup the **ToonyVoices** package. The error will look quite similar to the following, with the addition of a timestamp and showing you what file the error originated from:

```
IndexOutOfRangeException: Index was outside the bounds of the array.
```

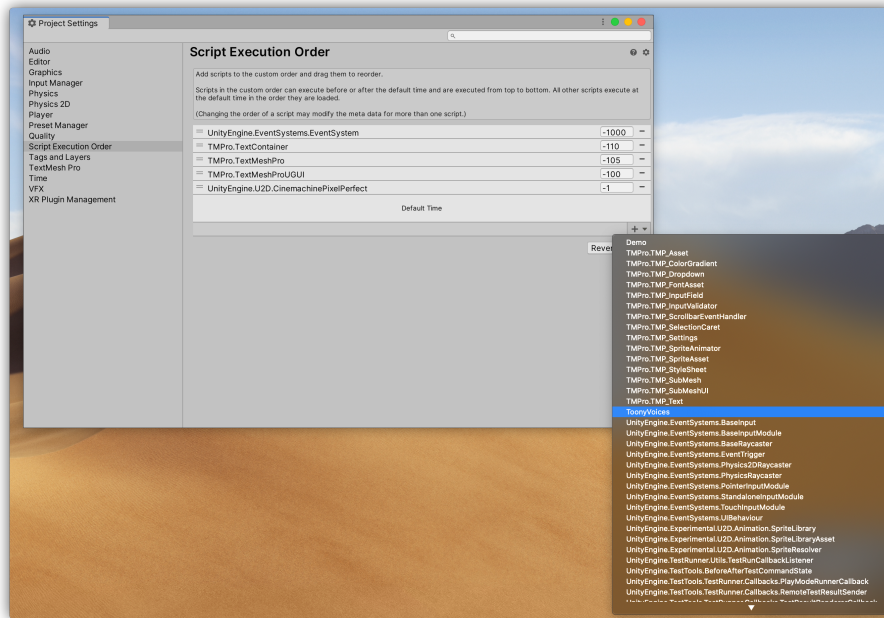
The cause of the exception is due to a script trying to use the `ToonyVocies` component before it has loaded all of its resources and built the `Dictionary`.

The fix for this error though is a simple one! We just need to tell Unity to load the **ToonyVoci**es script before our others.

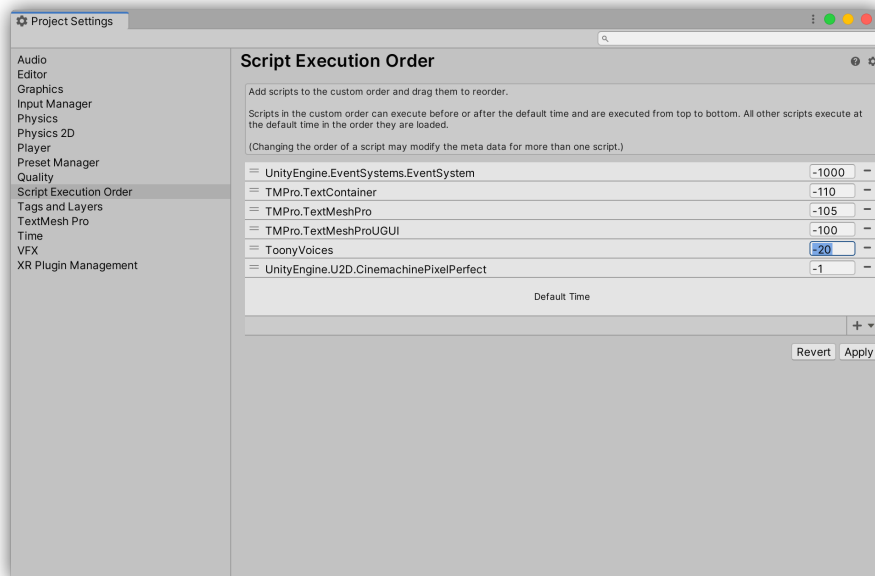
The settings panel to do this can be found from the toolbar **Edit > Project Settings..** and selecting **Script Execution Order** roughly half-way down on the left side of the new window.



From this window, you will want to find and click on the little plus symbol in the bottom right corner, and find the ToonyVoices script from within that *LONG* list of options.



The last step required of you to get rid of that pesky `IndexOutOfRangeException` error message, is to assign the `ToonyVoices` script a negative value so that it loads before your other scripts; a value such as **-20** should do nicely. Do not forget to click the apply button after setting your value!



Credits

- @KenneyNL for the assets used within the demo. Take a look at the other resources offered, definitely an amazing source for free game assets. (kenney.nl)
- Matt March for the initial idea and spark that allowed me to turn ToonyVoices into a reality on the Unity platform.